Tema 1.1:

Ciclo de vida del dato textual

Minería de textos, Grado en Inteligencia Artificial (GRIA), Universidad de Vigo
© Prof. David Candal Ventureira - 2026

Tabla de contenidos

01 Contexto

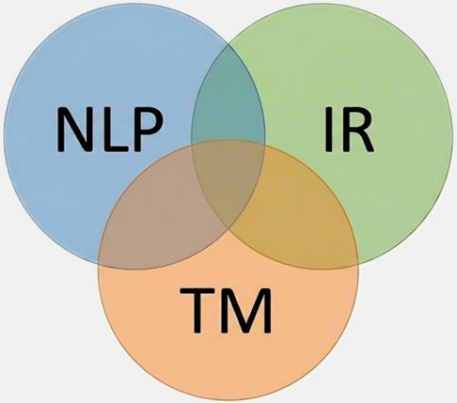
02 La naturaleza del dato textual

03 Procesamiento de datos textuales

04 Retos industriales

Contexto: TM vs NLP vs IR

- **NLP (Procesamiento del Lenguaje Natural):**
  - Estudia cómo analizar, comprender, modelar y generar el lenguaje humano, considerando su léxico, sintaxis, semántica y pragmática.
  - Ej: Traducción, análisis de sentimiento, reconocimiento de entidades…
- **IR (Recuperación de Información):**
  - Buscar y recuperar documentos o fragmentos relevantes para una consulta dentro de una colección grande de textos. Indexación y ranking.
  - Ej: Motores de búsqueda textual.
- **TM (Minería de Textos):**
  - Descubrir conocimiento (en base a patrones) a partir de un conjunto masivo de textos.
  - Diferentes aplicaciones: clasificación de documentos, resumen de textos, generación de texto…



Contexto: Evolución de TM/NLP

- **Minería clásica (1990-2012):**
  - Basada en frecuencias y co-ocurrencias (bolsa de palabras, BoW).
  - Comparación exacta y rígida.
- **Minería semántica (2013-2017):**
  - Basada en similitud semántica (embeddings de palabras y oraciones).
  - Comparación aproximada, pero adaptable (sinónimos).
- **“Minería/NLP generativa” (2018-actualidad):**
  - Comprensión, síntesis, generación y razonamiento lingüístico a través de redes neuronales (LLMs y RAG).
  - Se añade contexto semántico a cada palabra.
- **“Minería/NLP agéntica” (2023-actualidad):**
  - Autonomía en la toma de decisiones. El sistema decide qué texto extraer, cómo analizar el texto, planifica tareas e itera sobre ellas, y hace uso de herramientas externas.



La naturaleza del dato textual: Tipología I

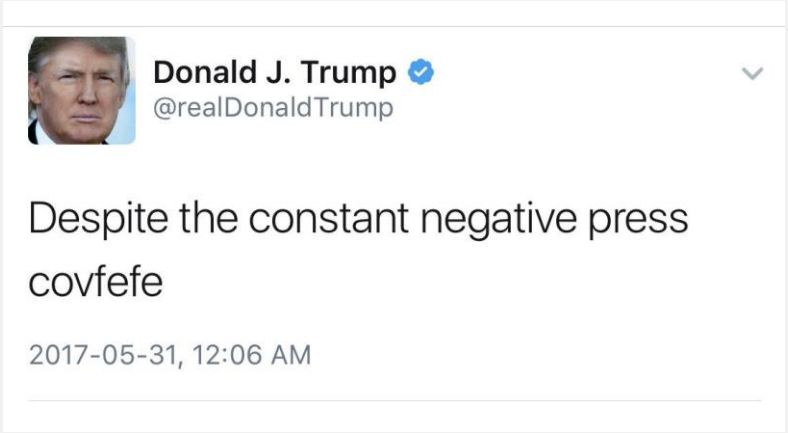
- **Según longitud:**
  - Textos cortos:
    - Tweets, comentarios…
    - Problema: Poca “señal” (información), falta contexto. Exceso de jergas, ironía, etc.
  - Textos largos:
    - Artículos, contratos legales…
    - Problema: Exceso de información -> alta relación ruido/señal. Requiere trocear el documento.
- **Según idioma:**
  - Documentos en diferentes idiomas.
  - Documentos que mezclan idiomas (incluso en una misma frase).

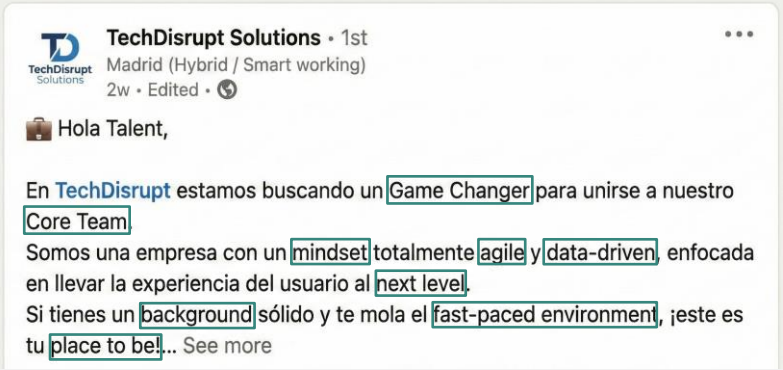




La naturaleza del dato textual: Tipología II

- **Según longitud e idioma:**
  - Ejemplos:

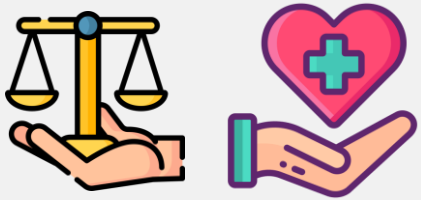




La naturaleza del dato textual: Tipología III

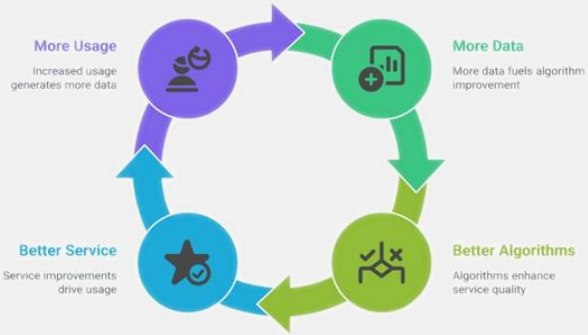
- **Según estructura:**
  - (Semi)estructurado:
    - JSON, CSV, YAML, XML, (HTML)…
    - Problema: Escenario poco frecuente en industria.
  - No estructurado:
    - PDFs, emails, logs, imágenes, audios…
    - Problema: Requieren procesado (y muchas veces conversión a texto).
- **Según dominio:**
  - Dominio general:
    - Wikipedia, enciclopedias…
  - Dominios específicos:
    - Legal, sanitario, financiero…
  - Problemas:
    - Falta de información relevante o alucinaciones si el dominio de los datos y la aplicación no coinciden.
    - Ambigüedad de dominio. Ej: “Acción” -> Hecho en dom. general, valor en dom. financiero, género en dom. cine…





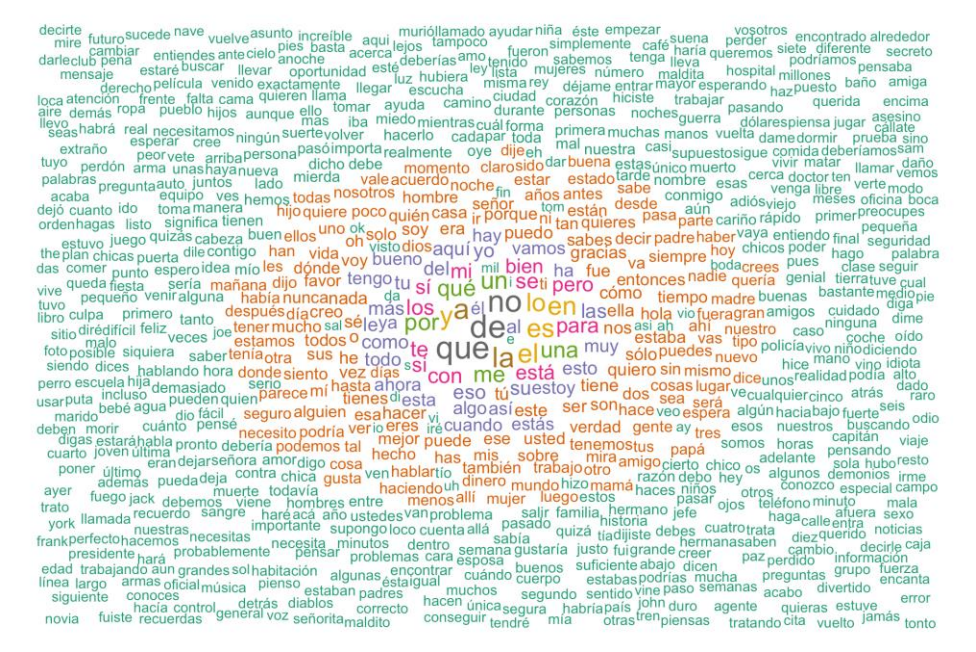
La naturaleza del dato textual: Producción I

- **Ley de Zipf (problema de la dispersión):**
  - El lenguaje sigue una distribución de potencia: unas pocas palabras irrelevantes aparecen muchas veces, muchas palabras relevantes aparecen muy pocas veces.
  - Problema: Necesitamos conjuntos muy grandes de datos para que el modelo pueda reconocer patrones relevantes.
- **“Garbage in, garbage out”:**
  - Un modelo alimentado con basura (datos mal preprocesados) produce basura (resultados poco fiables).
  - Una vez alcanzado cierto umbral de precisión, es más rentable centrarse en mejorar el corpus de entrenamiento que ajustar los hiperparámetros del modelo.
- **Data flywheel:**
  - Acumular datos de calidad permite entrenar mejores modelos.
  - Un mejor producto atrae más usuarios, lo que permite recoger mejores datos para su entrenamiento.



La naturaleza del dato textual: Producción II

- **Ley de Zipf:**
  - Ejemplo de conteo de frecuencias de palabras de subtítulos de películas en castellano (de: https://www.danielwells.me/posts/learning-languages-zipfs-law/).



Procesamiento de datos textuales

Fases de procesamiento en minería de texto:

1. Adquisición y almacenamiento de datos crudos.
2. Limpieza.
3. Preprocesamiento y vectorización.
4. Almacenamiento de datos procesados.
5. Explotación.



Procesamiento de datos textuales: Adquisición y almacenamiento de datos crudos I

Adquisición de datos crudos:

- Extracción de textos a partir de:
  - Bases de datos.
  - Dataset de documentos.
  - WEB scraping.
  - Conversión desde otros tipos de fichero: imagen (OCR), audio (STT)…
- Retos a la hora de extraer textos:
  - Documentos textuales (estructurados o no estructurados): Información fácilmente extraíble.
  - Documentos no textuales / basados en layout (.pdf, .docx, .pptx, …): Difíciles de procesar (segmentos de texto ubicados espacialmente, es complejo extraer el texto correctamente).

Procesamiento de datos textuales: Adquisición y almacenamiento de datos crudos II

Almacenamiento de datos crudos:

- Patrón WORM (write once, read many):
  - Construir un dataset de datos crudos base y **no modificarlo** (trabajar sobre una copia).
  - Esto nos permite recuperar los datos originales cuando actualizamos el proceso de limpieza y preprocesamiento.

- Bases de datos relacionales (SQL):
  - Diseñadas para transacciones, **no son óptimas para la lectura masiva de datos...**

Procesamiento de datos textuales: Adquisición y almacenamiento de datos crudos III

Estrategias de almacenamiento:

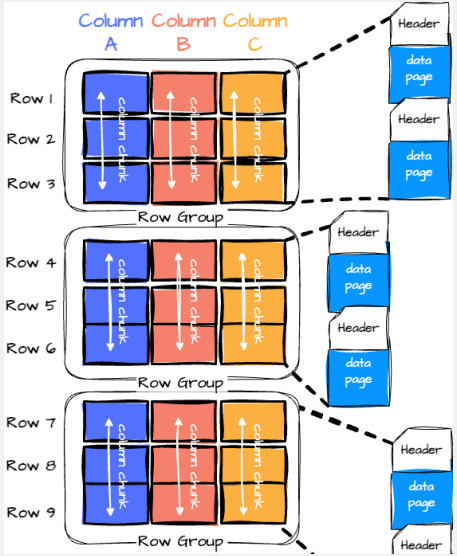
- JSONL:
  - Secuencia de JSONs.
  - Tolerante a fallos en el crawler y a entradas erróneas (ej: tipado no uniforme).
  - Lectura lenta (especialmente lecturas parciales).
- Apache Parquet:
  - Formato de fichero para el almacenamiento columnar y binario.
  - Rápida lectura de columnas individuales.
  - Compresión de tamaño.
  - Tipado estricto -> No tolerante a errores de escritura o formato.
- Híbrida:
  - JSONL para el almacenamiento tras adquisición de datos (tolerancia a fallos), Apache Parquet tras la limpieza (lectura selectiva).

Procesamiento de datos textuales: Adquisición y almacenamiento de datos crudos IV

Estrategias de almacenamiento (JSONL y Apache Parquet):


```
{"Name":"Alfred","Sex":"M","Age":14,"Height":69,"Weight":112.5}
{"Name":"John","Sex":"M","Age":12,"Height":59,"Weight":99.5}
{"Name":"Joyce","Sex":"F","Age":11,"Height":51.3,"Weight":50.5}
{"Name":"Judy","Sex":"F","Age":14,"Height":64.3,"Weight":90}
{"Name":"Louise","Sex":"F","Age":12,"Height":56.3,"Weight":77}
{"Name":"Mary","Sex":"F","Age":15,"Height":66.5,"Weight":112}
{"Name":"Philip","Sex":"M","Age":16,"Height":72,"Weight":150}
```

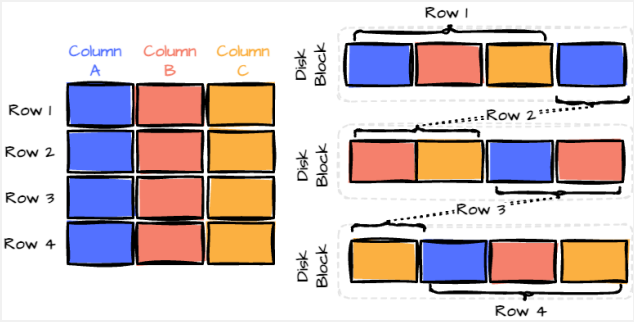

Ejemplo de fichero JSONL
(no es una lista de JSONs…)



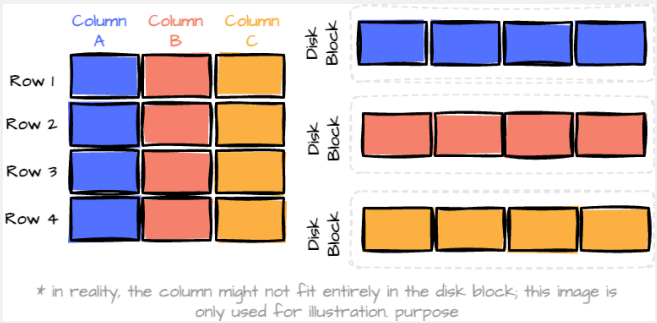
Ejemplo de fichero Parquet
(permite grupos de entradas)

Procesamiento de datos textuales: Adquisición y almacenamiento de datos crudos V

Estrategias de almacenamiento (almacenamiento por entradas vs por columnas):



Almacenamiento por entradas



Almacenamiento por columnas

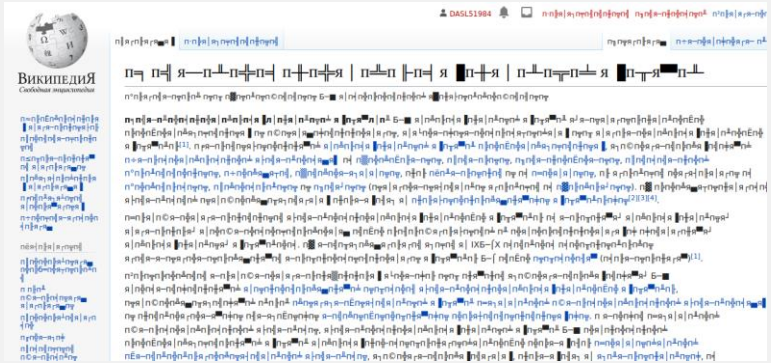
Procesamiento de datos textuales: Limpieza I

Tareas de limpieza:

- **Normalización de codificación**:
  - Los textos pueden estar codificados en diferentes formatos (UTF-8, ISO-8859-1, Windows-1252…). Normalización Unicode (NFC vs NFD).
  - Para poder preprocesarlos (considerar como igual una misma palabra codificada de diferente manera) debemos unificar la codificación (UTF-8).
  - Correción de mojibakes: caracteres incorrectos resultado de errores de decodificación.
- **Limpieza de boilerplate**:
  - Eliminar el ruido (texto no relevante) del documento: Menús, banners, anuncios (HTML), encacabezados y pies de página (artículos), etc.
- **Deduplicación**:
  - Eliminación de documentos que contienen el mismo texto.
    - Exacta: Detección de textos idénticos mediante funciones hash (ej: SHA256).
    - Difusa: Detección de textos equivalentes, con pequeñas variaciones (cambios en puntuación, párrafo adicional, etc).
  - La deduplicación es clave para evitar la contaminación entre los conjuntos de entrenamiento y prueba, mejorando la calidad y validez de los modelos.

Procesamiento de datos textuales: Limpieza II

Tareas de limpieza (ejemplos):



Ejemplo de mojibakes



Ejemplo de boilerplate

Procesamiento de datos textuales: Preprocesamiento y vectorización I

Tareas de preprocesamiento (NLP):

- Normalización lingüística básica: conversión a minúsculas, eliminación de signos de puntuación y símbolos especiales, eliminación de stopwords…
- Filtrado por idioma: pipelines independientes por idioma o descarte del texto.
- Tokenización:
  - Clásica: Tokenización por palabras + lematización o stemming (técnicas clásicas, CNNs/RNNs, y word2vec).
  - Moderna: Tokenización por subpalabras (Transformers y fastText).
- Feature augmentation: Añadir características adicionales a la matriz de características (ej: metadatos del documento, como longitud). Esta matriz se combina con la matriz de características.

Procesamiento de datos textuales: Preprocesamiento y vectorización II

Técnicas de vectorización (NLP):

- Transformación de texto en vectores o matrices numéricas para su procesamiento por modelos de aprendizaje automático:
  - **Modelado clásico**: Conteo de frecuencias de ocurrencia (BoW, TF-IDF).
  - **Embeddings**: Representación densa y continua de la información semántica del token/documento, en un espacio vectorial acotado (Word2Vec, BERT…).
  - **Grafos**: Capturan relaciones (semánticas, sintácticas o de co-ocurrencia, aristas del grafo) entre tokens/sentencias/documentos (nodos del grafo) (TextRank, Graph-of-Words…).
  - **Híbridos**: Combinación de alguno de los anteriores.

Procesamiento de datos textuales: Almacenamiento de datos procesados

Técnicas de almacenamiento de datos procesados:

- **BoW**:
  - CSV: Textual, pero no optimizado para lectura y almacenamiento.
  - Apache Parquet: Binario, optimizado para lectura y almacenamiento.
- **Embeddings**:
  - Embeddings de palabras: Parquet o objetos serializados.
  - Embeddings de documentos: Parquet o serialización, gestores como FAISS (ficheros), o DB vectoriales (optimizadas para consulta por similitud semántica).
- **Grafos**:
  - DB de grafos (optimizadas para consulta de relaciones).

Procesamiento de datos textuales: Explotación

Aplicaciones:

- Motores de búsqueda: Encontrar documentos/textos relacionados con una consulta.
  - Búsqueda literal (BoW).
  - Búsqueda semántica (embeddings).
- Modelado:
  - Clasificación (aprendizaje supervisado).
  - Clustering (aprendizaje no supervisado).
  - Resumen automático de textos (extractivo o abstractivo).
  - Sistemas de recomendación.
  - Generación (LLMs).

Procesamiento de datos textuales: Resumen



Retos industriales: Escalabilidad y coste

- Escalabilidad:
  - RAM: Trabajaremos (potencialmente) con corpus textuales gigantes -> streamming (iteradores) de datos durante procesado.
  - CPU: Por la misma razón -> paralelización de los procesos.
- Coste:
  - Modelos sencillos (NB, SVM, resumen extractivo) -> bajo coste computacional y menor latencia.
  - Modelos complejos (Transformers) -> resultados más complejos, pero alto coste computacional y mayor latencia.
  - Estrategia: Combinar ambos conjuntos de técnicas en base a cada tarea específica.

Retos industriales: Deriva temporal y legislación

- Deriva temporal:
  - Derivación semántica:
    - Las palabras adquieren y pierden contexto con el tiempo. Ejemplo:
      - En textos antiguos, corona se vincula a reinos.
      - Desde 2000s, surge el significado de bebida.
      - En 2019-2022, surge y desaparece en el contexto médico.
    - Problema: Se requiere ajuste semántico periódico.
  - Derivación temática:
    - Los temas también adquieren y pierden popularidad con el tiempo.
    - Problema: Clases pueden desbalancearse con el tiempo.
- Legislación:
  - Información personal identificable (PII): Nombre, DNI, correo…
  - GPDR (derecho al olvido): Eliminar datos personales de un modelo entrenado es muy complicado.
  - Estrategia: Eliminar PII antes de entrenar.

¿Preguntas?